Announcement: "Instal gitbash or Launch RedHat EC2…"
Reyaz Shaik
Created Jan 28Jan 28

Instal gitbash or Launch RedHat EC2 instance and login

```
sudo -s

mkdir gitproject

cd gitproject

yum install git -y

git -v

git status --> You get error, because this is not initialized yet

git init  --> initialize git , this is local repo. .git directory will be create which contain files/dir maintained by git. The .git directory contains metadata and the entire history of the project,

git status


touch index.html
--> it has created locally, now move from working directory to staging area

vi index.html
this is sample file

git status
--> this is not tracked, Not tracked files shows in red color, if you want to track file move to staging area

git add index.html
--> command to move to staging area

git status
--> now file is in staging area, tracked files shows green color, and now move to local repository

git commit -m "my first commit" index.html
--> now file is in local repository

git status

what ever we do track, committing file etc will be in .git directory

--> now another example to create a file and add to staging and repo

touch hello.txt
git status
git add hello.txt
git commit -m "My second commit" hello.txt
git status


touch test.txt
git status
git add test.txt
git commit -m "My third commit" test.txt
git status
```

```
--> create mode 100644  means a normal file
--> 100755 means executable file
----------------------------------------

git log --> to see how many commits history
git log --oneline --> to see less lines
git log --oneline -2 --> to see last 2 commits history
git show --> Show the changes of the last commit
git show 150eb87 --> to see which file is committed for this id

git log -p -2  --> shows last 2 commits with diff
git log --stat --> summary of the changes

git log    --pretty=oneline --> to see less info about commit details in
singleline
git log --pretty=format:"%h-%an,%ar:%s"   [h = hash/commitnumber, an =
authorname, ar = time, s = commit message ]
git shortlog --> Summarize git log output

vi index.html
this is first line

git add index.html
git commit -m "first line" index.html


git blame index.html --> Show who changed which line in a file

-------------------------------------
--> modify index.html file

vi index.html
this is second line .. new line
git add index.html
git commit -m "new line" index.html

Now see the difference between the commits

git log --oneline -2
git diff 150eb87..1fg0e787  -- see the diff between commits

Vi index.html
This is the third line

git diff  ------> compare changes working directory to staging area

git add index.html

git diff --staged --> compare changes staging to local repo

git commit -m "comparison" index.html

vi index.html
change something

git diff HEAD ----compare changes local directory to local repo [HEAD in caps]

-----------------------------------------

git log

--> you see root user in commit log --> in real time we should we our name not
root
```

This is User Configuration

```
git config user.name "reyaz"
git config user.email "reyaz@gmail.com"

git config user.name  [to see who is the user]
git config user.email
```

Another way of adding user configuration

```
git config --list
git config --global --edit
or
cd /root
cat .gitconfig

vi index.html
added a new line

git add index.html
git commit -m "added new line" index.html

git log --oneline --author='reyaz'  --- this show only committed from certain
person
git log --author="reyaz"

--> now create a new file
touch file1
git add file1
git commit -m "file1 commit" file1
git status
git log
--> now we can see Reyaz username

=========================
Git Amend - Change the commit history
=========================
if you want to change the last commit history

git commit --amend -m "an updated commit message"   --> this is replace latest
commit message

==================
git rebase - If you want to change multiple commit histories
==================
Note: if you want to use this command, you need to have minimum 3 commits in
your repo

git rebase -i HEAD~3

The option -i means interactive. HEAD~3 is a relative commit reference, which
means the last 3 commits from the current branch HEAD.

The git rebase command above will open your default text editor (Vim for
example) in an interactive mode showing the last 3 commits

Now replace pick with reword and keep changing the commit messages for 3 times
and wq!

git log --oneline -2   [All commit messages has been changed]


Squashing in Git is the process of combining multiple commits into a single,
```

consolidated commit. This is typically done to clean up the commit history before merging a feature branch into the main branch.

==========
Adding all files to stage at a time: git add .
==========

```
touch python{1..5}
git add --a   [to add all files at a time to staging]
git add . or git add *
git commit -m "python files" .
```

==========

==========

GIT Checkout : To restore previous version files in git

==========

clean up

```
--------
rm -rf *
git status
git add .
git commit -m "clean" .
```

```
vi index.html
This is version 1

git add .
git commit -m "version 1 commit" index.html

vi index.html
This is version 1
This is version 2

git add .
git commit -m "version 2 commit" index.html

vi index.html
This is version 1
This is version 2
This is version 3

git add .
git commit -m "version 3 commit" index.html

git log
or
git log --oneline -2

git show f86060:index.html
```

This will show first version file

If you want to restore previous version files

```
git checkout f86060 -- *  [* will restore all files in that commit]

git checkout f86060 -- index.html [It will restore only index.html]
```

```
cat index.html

Again if you want latest

git checkout master -- *  [coming back to latest file]

cat index.html

Another scenario of checkout
---------------------------
use git rm to remove the file and restore using checkout HEAD

vi test.html
this is test

git add test
git commit -m "test" test.html

git rm test.html --> this will remove the file

git checkout HEAD -- test.html ---> this will restore the file

=====================================================
GIT RESTORE :
git checkout and git restore does the same work: reverting to previous state

Scenario 1: if you want to undo change after saving and before commit
----------

How to undo changes after save. If you are working on latest file, you do
changes and saved. If you want previous one
use git restore index.html

vi test.html
This is demo

git add .
git commit -m "test" test.html

vi test.html
This is demo
ajffjkdhgdkjghdf
save it

cat test.html

if you want now to restore

git restore test.html

cat test.html

================================
 Scenario 2 = To restore files from stagged to unstage

Accidently you have added to stage using "git add" command and if you want to
unstage or untrack
=====================================================

How to untrack files

vi test.html
This is devops demo
```

```
git add test.html
git status --> now accidently i have stagged or tracked test.html , but I want
to untrack/unstage

git restore --staged test.html
(or)
git restore --staged . -->  [. all]
(or)
git rm --cached test.html --> same like restore command above but another
command to untrack/unstage

git status --> you can see now untracked

git add test.html  --> Add it to the stage again

git commit -m "test" test.html

Scenario - 3 : If you accidently delete the file from your directory and want to
get back
--------------

touch newfile
git add newfile
git status ---> it shows green color, it is stagged

git add newfile  ----> now stage it

rm -rf newfile  -----> delete the file locally

ls  ---> no local file

git restore newfile   --> this will help to get the deleted file back from stage
to local machine

git status

git commit -m "newfile" newfile


==================
GIT RESET --- Uncommit last commit [if you committed accidentally and want to
uncommit or come the file back to local repo to staging]
or
Unstage files that were mistakenly added to the staging area.

vi Reyaz.html
Welcome to DevOps Classes
git add Reyaz.html
git commit -m "reyaz" Reyaz.html


vi Reyaz.html
Welcome to DevOps Classes
ijkdfhgdkfghdkjfhgjdfhg

git add Reyaz.html
git commit -m "reyaz" Reyaz.html

oh shit, i commit wrong code

Accidentally you commited now, but you want to get it back from local repo to
staging
```

git status --> all clean, nothing to commit

git reset --soft HEAD^   [uncommit and keep the changes]

git status -- we see Reyaz.html now in stage

again git add Reyaz.html
git commit -m "reyaz" Reyaz.html
now try below

git reset --hard HEAD^   [uncommit and discard the changes]
cat Reyaz.html

Feature	git restore	git reset
Scope	Working directory or staging area	Commit history, staging area, working directory
Affects Commit History	No	Yes (can modify HEAD)
Primary Use Case	Undo changes or unstage files	Move HEAD or reset state to a previous commit
Safety	Less destructive	Can be destructive, especially with --hard

HEAD is a pointer to the current branch or commit you are working on
In Git, HEAD is a source to the current branch or commit you are working on. HEAD normally shows the recent commit of the current branch and moves when you switch branches or check out exact commits.

git Restore: Does not modify commit history or the repository's HEAD pointer. It only reverts changes in the working directory or staging area.

git reset: Can modify commit history by moving the HEAD pointer and potentially discarding changes.

==========
help
===========

git help

==========
GIT STASH -- If you want to hide/park all the changes you did on the repo locally to a tmp place and proceed with another story changes then stash the old story, explain in diagram

The git stash command is a useful feature in Git that allows you to temporarily save changes in your working directory without committing them

==========
rm -rf *
git add .
git commit -m "clean" .

Now create a story1 file

```
------------------------
vi story1
 i am working on story 1
 very difficult task
 it task 2 hours time

--> Now suddenly manager called and asked to work on story2

git status
git add story1

git stash --> story1 will be removed from the working dir, do ls, it will be
placed in tmp location

ls -- story1 file is not there

git status --> working tree clean

using stash we kept story 1 aside

Now create story2 file
---------------------
vi story2
i need to complete this story2 first

git status
git add story2
git commit -m "story2" story2

git status

Now lets work back on story1
---------------------------

git stash apply --> now restore do ls
ls
git status -> you can see the file again

git stash list -- it will list the stashes
git stash clear -- it will clear the stash


if you want to stash multiple files

git stash push -m "commit message" -- story1 story2
git status

Announcement: "================================…"
Reyaz Shaik
```